

ENGLISH For INTERVIEW

Introduce

[EN] Hello, I'm Khoi Nguyen. I'm a Fullstack Developer with over five years of experience, and most of my work has focused on backend development.

I mainly work with Python and Golang. I've worked on both monolithic and microservices systems, including backend services that need to handle growing traffic. I also have experience with AWS services like EC2, Lambda, SNS, and SQS. I'm familiar with Docker and CI/CD, so I can support deployment and basic system operations when needed.

On the frontend side, I mainly use React. Even though frontend is not my main focus, working as a fullstack developer helps me understand how the whole system works and makes it easier to work with other team members.

In previous projects, I developed new features, improved and cleaned up APIs, optimized database queries, and worked with clients or third-party teams for integration tasks. I enjoy solving technical problems and improving system stability and performance step by step. Currently, I'm looking for a new environment where I can continue learning and growing as a backend or fullstack engineer, especially in projects that build scalable systems and AI integration—one of the most powerful and defining technologies today.

[VN] Chào anh/chị, em là Khôi Nguyễn. Em là Fullstack Developer với hơn 5 năm kinh nghiệm, trong đó em làm backend nhiều hơn.

Em làm việc chủ yếu với Python và Golang. Em đã tham gia phát triển cả hệ thống monolith và microservices, trong đó có các hệ thống cần khả năng mở rộng. Em cũng có kinh nghiệm làm việc với các dịch vụ AWS như EC2, Lambda, SNS và SQS. Ngoài ra, em quen với Docker và quy trình CI/CD nên có thể tham gia vào việc triển khai và vận hành hệ thống khi cần.

Về frontend, em sử dụng React là chính. Dù không tập trung hoàn toàn vào frontend, nhưng nhờ làm fullstack nên em hiểu được luồng xử lý tổng thể của hệ thống và phối hợp với các thành viên khác trong team thuận lợi hơn.

Trong các dự án trước, em tham gia phát triển tính năng mới, chỉnh sửa và cải thiện API, tối ưu truy vấn database, cũng như trao đổi với khách hàng hoặc bên thứ ba khi có yêu cầu tích hợp. Em thích xử lý các vấn đề kỹ thuật và cải thiện dần chất lượng hệ thống qua từng giai đoạn.

Hiện tại, em đang tìm một môi trường mới để tiếp tục học hỏi và phát triển thêm theo hướng backend hoặc fullstack, đặc biệt là các hệ thống có định hướng mở rộng và tích hợp công nghệ AI — vốn đang là một trong những xu hướng công nghệ mạnh mẽ nhất hiện nay.

Dự án gần nhất, thích nhất, khó khăn nhất, cách giải quyết khó khăn

[EN] One of the projects I found both interesting and challenging — and also my most recent one — was a Lodging and Transport Management System.

It was a web-based system used to manage hotel bookings and transportation for business trips. The system handled things like booking validation, integration with external partners, sending automatic notifications, and synchronizing data between different services.

What made this project interesting to me was that it was built using a microservices architecture combined with an event-driven pattern. Instead of putting everything into one big system, we separated features into different services, and they communicated with each other through events. This made the system more flexible and easier to scale, but it also made things more complex.

The biggest challenge was debugging. In a monolithic system, when something goes wrong, you usually check one place and can find the issue. But in a microservices and event-driven system, one user action can create multiple events that go through different services. Sometimes the error showed up in Service A, but the real problem was actually in Service C, where an event was handled incorrectly. So debugging required checking logs across multiple services and clearly understanding the event flow.

To handle this, I spent time learning the overall architecture and how the services communicated with each other. I also improved logging in some parts of the system to make troubleshooting easier. After working on it for a while, I became more comfortable tracing event flows and finding the root cause faster.

Another challenge was optimizing database queries. Some booking and reporting features had complex queries with multiple joins and filters. As the data grew, we started to notice slower response times.

To improve performance, I analyzed the execution plans of slow queries to find bottlenecks. In some cases, I added proper indexes. In other cases, I rewrote the queries to reduce unnecessary joins or split large queries into smaller ones. After these changes, the response time became more stable and noticeably faster.

I was also involved in integrating AWS services like SQS and SNS for handling events. During that process, I had to pay attention to message retries and avoid duplicate processing. This helped me better understand how distributed systems work and how to design them more carefully.

Overall, this project helped me learn a lot about system design, distributed systems, performance tuning, and troubleshooting in a production environment. It's a project that helped me grow quite a bit technically.

[VN] Một trong những dự án mà em thấy vừa thú vị vừa khó khăn nhất cũng là dự án gần nhất của em, nó là về Lodging và Transport. Đây là một hệ thống web dùng để quản lý việc đặt khách sạn và phương tiện di chuyển cho các chuyến công tác. Hệ thống có nhiều

chức năng như xác nhận booking, tích hợp với đối tác bên ngoài, gửi thông báo tự động và đồng bộ dữ liệu giữa các service.

Điều em thấy thú vị là hệ thống được xây dựng theo kiến trúc microservices kết hợp với event-driven pattern. Thay vì tất cả logic nằm trong một hệ thống lớn, mỗi chức năng được tách thành các service riêng và giao tiếp với nhau thông qua event. Điều này giúp hệ thống dễ mở rộng và linh hoạt hơn, nhưng đồng thời cũng làm tăng độ phức tạp.

Khó khăn lớn nhất là việc debug. Trong hệ thống monolith thì khi có lỗi, mình thường trace ở một chỗ là có thể tìm ra nguyên nhân. Nhưng với microservices và event-driven, một thao tác của người dùng có thể tạo ra nhiều event, đi qua nhiều service khác nhau. Có những lúc lỗi xuất hiện ở Service A nhưng nguyên nhân thật sự lại nằm ở Service C do xử lý event chưa đúng. Vì vậy, để debug thì phải kiểm tra log ở nhiều service và hiểu rất rõ luồng xử lý của hệ thống.

Để xử lý vấn đề này, em dành thời gian tìm hiểu kỹ hơn về kiến trúc tổng thể và cách các service giao tiếp với nhau. Em cũng chú ý cải thiện log ở một số phần để việc theo dõi và phân tích lỗi dễ hơn. Sau một thời gian làm quen, em đã quen hơn với việc trace theo event flow và xác định được nguyên nhân gốc của vấn đề nhanh hơn.

Ngoài ra, một khó khăn khác là tối ưu truy vấn database. Một số chức năng liên quan đến booking và báo cáo có truy vấn khá phức tạp, join nhiều bảng và lọc theo nhiều điều kiện. Khi dữ liệu tăng lên, hệ thống bắt đầu có dấu hiệu chậm.

Để cải thiện, em phân tích execution plan của các query chậm để tìm điểm nghẽn. Có trường hợp em thêm index phù hợp, có trường hợp em viết lại query để giảm join không cần thiết hoặc chia nhỏ truy vấn. Sau khi tối ưu, thời gian phản hồi ổn định hơn và cải thiện rõ rệt.

Ngoài ra, em cũng tham gia tích hợp các dịch vụ AWS như SQS và SNS để xử lý event. Trong quá trình đó, em phải chú ý đến việc retry message và tránh xử lý trùng lặp, điều này giúp em hiểu rõ hơn về cách thiết kế hệ thống phân tán.

Tổng thể, dự án này giúp em học được nhiều về system design, xử lý hệ thống phân tán, tối ưu hiệu năng và troubleshooting trong môi trường production. Đây là dự án khiến em phát triển hơn khá nhiều về mặt kỹ thuật.

Dự án gần nhất có tích hợp AI

[EN] Yes, I was involved in an AI-related project before. It's not listed in my CV because I joined during the early stage and mainly worked on the service structure and system setup. The project was about using AI to support agricultural product traceability. Each product had a QR code attached. When users scanned the QR code, the system could provide detailed information about the product, starting from when it was still at the farm until it reached the market.

For example, it could show pesticide spraying dates, harvesting dates, packaging dates, and packaging location. It could also provide more operational information, such as

whether similar products were currently on store shelves, in a warehouse, or at a specific nearby location. Users could ask these questions through an AI chatbot. The product also included different portals for stakeholders like farms, factories, supermarkets, distributors, and logistics teams.

In this project, my main responsibility was building the basic structure of the microservices and setting up CI/CD. I developed the services using Golang. I built a service called Chat Hub, which worked as a communication layer between the AI bots and client applications, and it also stored chat history.

I also supported the AI team in deploying their chatbot services built with Rasa framework. I helped containerize them using Docker and set up Jenkins for deployment.

I didn't directly train AI models, but I worked on the system architecture, integration, and deployment side of the AI solution. This project helped me understand better how AI systems can be integrated into real-world products and run in production.

[VN] Dạ, em có từng tham gia một dự án có liên quan đến AI, tuy nhiên dự án này không có trong CV vì em tham gia ở giai đoạn đầu và chủ yếu phụ trách phần structure code của các service và triển khai hệ thống.

Dự án đó là về việc sử dụng AI để hỗ trợ truy xuất nguồn gốc nông sản. Mỗi sản phẩm sẽ được gắn một mã QR. Khi người dùng quét QR, hệ thống có thể cung cấp các thông tin liên quan đến sản phẩm, từ lúc còn ở vườn cho đến khi ra thị trường.

Ví dụ như ngày phun thuốc, ngày thu hoạch, ngày đóng gói, nơi đóng gói. Ngoài ra còn có các thông tin vận hành chi tiết hơn, như sản phẩm tương tự hiện đang ở trên kệ, trong kho hay ở một địa điểm cụ thể nào đó. Người dùng có thể hỏi các thông tin này thông qua một chatbot AI. Hệ thống cũng có các portal dành cho nhiều bên liên quan như nông trại, nhà máy, siêu thị, nhà phân phối và logistics.

Trong dự án này, nhiệm vụ chính của em là xây dựng cấu trúc nền tảng cho các microservices và thiết lập CI/CD. Em phát triển các service bằng Golang. Em có xây dựng một service gọi là Chat Hub, đóng vai trò trung gian giao tiếp giữa các AI bot và client, đồng thời lưu trữ lịch sử chat.

Ngoài ra, em cũng hỗ trợ team AI trong việc triển khai các chatbot được xây dựng bằng Rasa, đóng gói bằng Docker và triển khai qua Jenkins.

Em không trực tiếp huấn luyện model AI, nhưng em tham gia vào phần kiến trúc hệ thống, tích hợp và triển khai giải pháp AI vào môi trường thực tế. Qua dự án này, em hiểu rõ hơn cách các hệ thống AI có thể được đưa vào sản phẩm thật và vận hành trong thực tế.

Optimize Python Docker Image

[EN] Yes, I have been using Docker in my projects for packaging and deploying backend services.

At the beginning, I used a simple Dockerfile, similar to a single-stage build. It was based on the official Python image, and I installed dependencies directly inside it using pip or poetry. It worked fine, but I noticed that the image size was quite large and the build time was slow, especially when rebuilding in CI/CD.

After some time, I started researching how to optimize Docker images. I read articles, GitHub discussions, and DevOps blogs. From there, I learned about multi-stage builds and image hardening techniques.

So I refactored my Dockerfile to use multi-stage build. In the first stage, I installed and built all dependencies. In the second stage, I only copied the minimal runtime files needed to run the application. This helped reduce the image size significantly because I didn't include build tools, caches, or unnecessary packages in the final image.

To optimize further, I switched to a smaller and more secure base image, like distroless instead of a full Debian-based Python image. Distroless does not include a shell or package manager, so it reduces the attack surface and makes the container more secure.

I also made sure the container runs as a non-root user, which is a security best practice.

To speed up build time, I used Docker BuildKit cache mount for dependency installation.

That way, dependencies are cached between builds, and CI pipelines become much faster.

For security, I pinned image versions using SHA digest to avoid unexpected changes. I also checked for vulnerable dependencies and upgraded packages when needed.

So overall, I improved my Docker image in three main aspects:

- Smaller size using multi-stage build and minimal base image
- Faster build time using caching
- Better security by using non-root user, distroless image, and dependency scanning

Through this process, I learned that Docker optimization is not only about making the image smaller, but also about performance and security in production environments.

[VN] Có, tôi có sử dụng Docker trong các dự án backend của mình để đóng gói và deploy service.

Ban đầu tôi dùng một Dockerfile khá đơn giản, kiểu single-stage build, dựa trên image Python chính thức và cài dependency trực tiếp bằng pip hoặc poetry. Nó chạy được, nhưng tôi nhận thấy image khá lớn và thời gian build hơi lâu, đặc biệt là khi build lại trong CI/CD.

Sau đó tôi bắt đầu tìm hiểu thêm cách tối ưu Docker image. Tôi đọc blog, GitHub, các diễn đàn DevOps để xem best practice. Từ đó tôi biết đến multi-stage build và các kỹ thuật hardening image.

Tôi đã refactor lại Dockerfile để dùng multi-stage build. Ở stage đầu tiên tôi cài và build toàn bộ dependency. Ở stage thứ hai, tôi chỉ copy những file cần thiết để chạy ứng dụng. Cách này giúp giảm kích thước image khá nhiều vì không mang theo tool build, cache hoặc package không cần thiết.

Sau đó tôi tối ưu thêm bằng cách dùng base image nhỏ và an toàn hơn, ví dụ như distroless thay vì dùng full Python Debian image. Distroless không có shell hay package manager nên giảm attack surface và an toàn hơn.

Tôi cũng cấu hình để container chạy bằng non-root user, vì đó là best practice về security. Để tăng tốc build, tôi sử dụng Docker BuildKit cache mount cho phần cài dependency, giúp CI build nhanh hơn đáng kể.

Về bảo mật, tôi pin version image bằng SHA digest để tránh image bị thay đổi ngoài ý muốn, và tôi cũng kiểm tra lỗi hỏng dependency để upgrade khi cần.

Tóm lại, tôi tối ưu Docker image theo 3 hướng chính:

- Giảm kích thước bằng multi-stage và base image tối giản
- Tăng tốc build bằng cache
- Tăng bảo mật bằng non-root user, distroless và kiểm soát dependency

Qua quá trình đó tôi hiểu rằng tối ưu Docker không chỉ là làm image nhỏ hơn, mà còn liên quan đến performance và security trong môi trường production.

Optimize React Docker Image

[EN] Yes, I have experience optimizing Docker images for frontend applications, especially React projects.

At the beginning, I used a normal multi-stage build. In the first stage, I built the React app using Node. In the final stage, I used an nginx alpine image to serve the static files. This already reduced the image size compared to a single-stage build, and it worked well.

However, I wanted to optimize it further. I noticed that even nginx alpine still includes many packages that are not really needed for serving static files. So I started researching how to make the image even smaller, faster, and more secure.

After researching, I improved my Dockerfile in several ways.

First, I kept the multi-stage build but optimized dependency installation using pnpm with cache mount. By using Docker BuildKit cache, the dependency layer is reused between builds, so CI build time is much faster.

Second, instead of using the default nginx image, I built a fully static nginx binary from source. I enabled only the modules I needed, like SSL and HTTP/2, and disabled many unnecessary modules such as proxy, fastcgi, mail, and stream modules. This significantly reduced the binary size and attack surface.

Then I stripped debugging symbols and compressed the nginx binary using UPX to reduce the size even more.

Third, instead of using nginx alpine as the final image, I used FROM scratch. This means the final image contains only:

- The nginx binary
- The minimal filesystem
- The static React build files

There is no shell, no package manager, no OS utilities. This makes the image extremely small, less than 6MB, and very secure.

I also created a minimal user system manually and ran the container as a non-root user, which is important for security best practice.

In addition, I pre-compressed static assets using gzip and brotli during build time. So at runtime, nginx can directly serve pre-compressed files, improving performance and reducing CPU usage.

Finally, I pinned all base images using SHA256 digest to prevent supply chain attacks and ensure reproducible builds.

So overall, I improved the Docker image in three aspects:

- Smaller size by using scratch and static nginx
- Faster build using cache mount and optimized dependency layers
- Stronger security by removing unnecessary modules, running non-root, and reducing attack surface

Through this process, I learned that Docker optimization is not just about reducing megabytes, but also about performance, security, and production reliability.

[VN] Có, tôi có kinh nghiệm tối ưu Docker image cho frontend, đặc biệt là dự án React. Ban đầu tôi dùng multi-stage build cơ bản. Stage đầu build React bằng Node, stage cuối dùng nginx alpine để serve static file. Cách này đã tốt hơn single-stage và giảm được size image.

Tuy nhiên tôi thấy nginx alpine vẫn còn khá nhiều thành phần không cần thiết cho việc chỉ serve static file. Vì vậy tôi bắt đầu research thêm cách làm image nhẹ hơn, build nhanh hơn và an toàn hơn.

Sau khi tìm hiểu, tôi cải tiến Dockerfile theo nhiều hướng.

Thứ nhất, tôi tối ưu phần cài dependency bằng pnpm và sử dụng cache mount của Docker BuildKit. Nhờ đó dependency được cache giữa các lần build, giúp CI build nhanh hơn đáng kể.

Thứ hai, thay vì dùng nginx image có sẵn, tôi tự build một nginx static binary từ source. Tôi chỉ bật những module cần thiết như SSL và HTTP/2, và tắt rất nhiều module không cần như proxy, fastcgi, mail... Điều này giúp giảm kích thước binary và giảm attack surface.

Sau đó tôi strip symbol debug và nén binary bằng UPX để giảm size thêm.

Thứ ba, thay vì dùng nginx alpine làm image cuối, tôi dùng FROM scratch. Nghĩa là image cuối cùng chỉ chứa:

- nginx binary
- filesystem tối thiểu
- static React build

Không có shell, không có package manager, không có OS utility. Image rất nhỏ, dưới 6MB, và rất an toàn.

Tôi cũng tạo user tối thiểu và chạy container bằng non-root user để đảm bảo security best practice.

Ngoài ra, tôi nén sẵn static asset bằng gzip và brotli trong quá trình build để nginx có thể serve trực tiếp file đã nén, giúp tăng performance và giảm CPU runtime.

Cuối cùng, tôi pin version image bằng SHA256 để tránh rủi ro supply chain và đảm bảo reproducible build.

Tổng lại, tôi ưu tiên theo 3 hướng:

- Giảm kích thước bằng scratch và static nginx
- Tăng tốc build bằng cache mount
- Tăng bảo mật bằng giảm module, non-root và giảm attack surface

Qua đó tôi hiểu rằng tối ưu Docker không chỉ là làm image nhỏ, mà còn liên quan đến performance và security trong production.

Kubernetes

[EN] In my project, the Horizontal Scaling Gateway for Core Banking in Thailand, the customer was using Red Hat OpenShift, which is an enterprise version of Kubernetes. So, we were working on OpenShift, but the core concepts are the same as Kubernetes.

The DevOps team prepared everything very carefully. They gave us detailed documents and ready-made YAML files for all the services. My team just needed to review those YAML files, update the configurations based on the microservices we developed — like authentication, transaction routing, sync service, and data aggregation — and then push the updated files to GitLab.

After that, we triggered the deployment through Jenkins. Jenkins would run the pipeline, apply the YAML files to OpenShift, and deploy the services automatically. This way, everything was consistent, version-controlled in GitLab, and deployed safely without manual kubectl commands.

The main Kubernetes resources we used were:

- Deployment to manage our microservices pods, handle scaling, and do rolling updates without downtime.
- Service (mostly ClusterIP) for internal communication between services, so they could call each other reliably.
- Ingress as the entry point from outside, with path-based routing and HTTPS support.
- ConfigMap and Secret for configurations and sensitive data like database credentials or Kafka keys.
- Horizontal Pod Autoscaler (HPA) to automatically scale pods up or down based on CPU usage during high transaction loads.

- And sometimes PersistentVolumeClaim (PVC) with StorageClass if a service needed persistent storage, like for Redis.

Kubernetes — or OpenShift — helped us achieve automatic scaling, stable internal networking, secure configs, and reliable deployment through CI/CD. It was perfect for handling variable transaction volumes in a core banking system.

[VN] Trong dự án Horizontal Scaling Gateway cho Core Banking tại Thái Lan, khách hàng sử dụng Red Hat OpenShift — đây là phiên bản doanh nghiệp của Kubernetes. Nên em làm việc trên OpenShift, nhưng các khái niệm cốt lõi thì giống Kubernetes thông thường.

Đội DevOps chuẩn bị rất chi tiết: họ cung cấp đầy đủ tài liệu và các file YAML sẵn có cho tất cả các dịch vụ. Nhiệm vụ của team em là xem lại các file YAML đó, chỉnh sửa và config thông tin phù hợp với các microservice mà chúng em phát triển — như authentication, transaction routing, sync service, data aggregation — rồi push file lên GitLab.

Sau khi push, chúng em trigger pipeline trên Jenkins. Jenkins sẽ chạy tự động, apply các file YAML vào OpenShift và deploy dịch vụ. Quy trình này đảm bảo mọi thứ nhất quán, được version control trên GitLab, và deploy an toàn mà không cần chạy lệnh thủ công.

Các tài nguyên Kubernetes chính em sử dụng là:

- Deployment để quản lý pod của các microservice, scale theo chiều ngang và rolling update không downtime.
- Service (chủ yếu ClusterIP) để các service giao tiếp nội bộ ổn định qua DNS.
- Ingress làm điểm vào từ bên ngoài, với routing dựa trên path và hỗ trợ HTTPS.
- ConfigMap và Secret để lưu cấu hình không nhạy cảm và thông tin bí mật như password database hay credential Kafka.
- Horizontal Pod Autoscaler (HPA) để tự động tăng/giảm số pod dựa trên CPU khi tải transaction tăng cao.
- Và đôi khi PersistentVolumeClaim (PVC) với StorageClass nếu service cần lưu trữ bền vững, như Redis.

Kubernetes — hay OpenShift — giúp hệ thống tự động scale, giao tiếp nội bộ ổn định, cấu hình an toàn và deploy đáng tin cậy qua CI/CD. Rất phù hợp cho core banking cần high availability và xử lý tải transaction biến động.